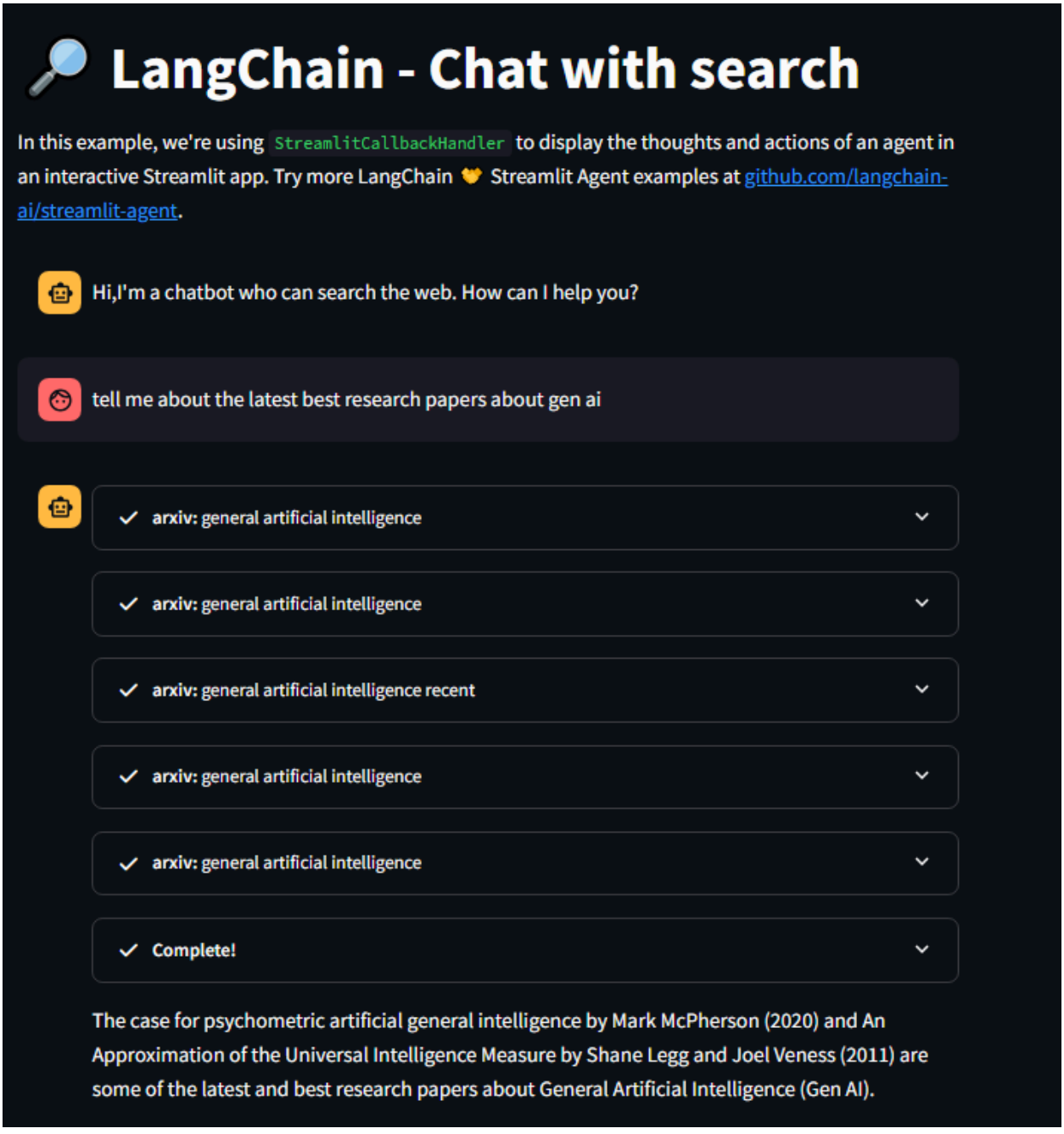
Search Engine GEN AI App




```
pip install -U duckduckgo-search
```

- **If you get rate limit error, use the following version 👇**

```
pip install -U duckduckgo_search==5.3.1b1
```

Import libraries:

```
import streamlit as st
from langchain_groq import ChatGroq
from langchain_community.utilities import ArxivAPIWrapper,WikipediaAPIWrapper
from langchain_community.tools import ArxivQueryRun,WikipediaQueryRun,DuckDu
ckGoSearchRun
from langchain.agents import initialize_agent,AgentType
from langchain_community.callbacks.streamlit import StreamlitCallbackHandler
```

`DuckDuckGoSearchRun` → Makes you search anything on the internet

`initialize_agent` →

- This is a **convenience function** to create a working LangChain **agent** in one line.
- It helps you build an agent by just specifying:
  - the **LLM**
  - the **tools**
  - the **type** of agent you want (via `AgentType`)

- **What it is:** This is a convenience function that helps you set up an agent quickly. Before the advent of more modular and flexible approaches like `create_openai_tools_agent` and LCEL (LangChain Expression Language), `initialize_agent` was the go-to way to create various types of agents.
- **How it works (generally):** You typically provide it with:
  - An `LLM` (the language model that acts as the agent's brain).
  - A list of `tools` (the external functionalities the agent can use).
  - An `agent` type (which determines the underlying reasoning framework).
  - Other optional parameters like `verbose=True` (to see the agent's thought process) or `handle_parsing_errors=True`.

- **Purpose:** It abstracts away the boilerplate code needed to set up an agent, including creating the necessary prompt, output parser, and runnable.
- **Current Status/Usage:** While still functional, `initialize_agent` is generally considered **legacy** for most new agent development, especially when using OpenAI models. T**he newer `create_openai_tools_agent` (for OpenAI function-calling models) and directly building agents using LCEL with `RunnableAgent` offer more flexibility, better control, and are often more performant.** However, `initialize_agent` is still useful for certain older agent types or when you want a very quick setup.

✅ What is `AgentType`?

`AgentType` is an **enum** (a list of predefined agent strategies) that tells `initialize_agent` what style of agent to use.

Common types:

AgentType	Description
`ZERO_SHOT_REACT_DESCRIPTION`	Agent uses tool names + descriptions to pick actions (basic ReAct agent).
`OPENAI_FUNCTIONS`	Uses OpenAI's tool-calling format (function-calling) with GPT-3.5/4.
`STRUCTURED_CHAT_ZERO_SHOT_REACT`	Uses structured tool input/output with ReAct.
`CHAT_CONVERSATIONAL_REACT_DESCRIPTION`	ReAct + memory support for chat models.

✅ Example usage:

```
from langchain.agents import initialize_agent, AgentType

agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.OPENAI_FUNCTIONS,
    verbose=True
)
```

So this gives you a **ready-to-use agent** based on what kind you want (OpenAI tool agent, chat agent, etc.).

`from langchain.callbacks import StreamlitCallbackHandler`

This imports a **callback handler** for **Streamlit apps**.

It lets LangChain **stream output directly to the Streamlit interface**.

✅ What it does:

- Captures intermediate agent actions (e.g., tool calls)
- Streams those steps (and final answers) to your Streamlit app
- Adds transparency and interactivity

What it is:

- A specialized callback handler specifically designed to work with the Streamlit web application framework.
- **How it works:** When you create a Streamlit app and integrate this handler into your LangChain agent or chain, it automatically:
  - **Displays intermediate steps:** As the agent reasons, calls tools, and gets observations, `StreamlitCallbackHandler` can render these steps directly in your Streamlit app's UI. This provides great transparency for the user.
  - **Streams LLM output:** If the LLM is generating a long response, the handler can stream the tokens as they are generated, making the application feel more responsive.
  - **Formats output nicely:** It typically formats the various steps (entering/exiting chains, tool calls, tool outputs, LLM responses) in a user-friendly way within the Streamlit UI.
- **Purpose:** To provide a rich, interactive, and transparent user experience for LangChain applications built with Streamlit, showing the "thought process" of the AI rather than just the final answer.

Arxiv, wikipedia & DuckDuckGo Tools

```
## Arxiv and wikipedia Tools
arxiv_wrapper=ArxivAPIWrapper(top_k_results=1, doc_content_chars_max=200)
arxiv=ArxivQueryRun(api_wrapper=arxiv_wrapper)

api_wrapper=WikipediaAPIWrapper(top_k_results=1,doc_content_chars_max=200)
wiki=WikipediaQueryRun(api_wrapper=api_wrapper)

search=DuckDuckGoSearchRun(name="Search")
```

`doc_content_chars_max` → The maximum character length for the document content

Create Streamlit App

```
st.title("🔎 LangChain - Chat with search")
"""
In this example, we're using `StreamlitCallbackHandler` to display the thoughts and actions of an agent in an interactive Streamlit app.
Try more LangChain 🤝 Streamlit Agent examples at [github.com/langchain-ai/streamlit-agent](https://github.com/langchain-ai/streamlit-agent).
"""

## Sidebar for settings
st.sidebar.title("Settings")
api_key=st.sidebar.text_input("Enter your Groq API Key:", type="password")
```

Create a session state

```
if "messages" not in st.session_state:
    st.session_state["messages"]=[
        {"role":"assistant","content":"Hi,I'm a chatbot who can search the web. How can I help you?"}
    ]
```

```
for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg['content'])
```

`st.session_state` :

- This is a core feature of Streamlit. It's a Python dictionary-like object that allows you to store and access variables **across reruns of your Streamlit app**.
- Streamlit apps rerun from top to bottom every time a user interacts with a widget (e.g., types in a text box, clicks a button) or when the code changes.
- `st.session_state` is crucial for maintaining state (like chat history, user selections, etc.) between these reruns, otherwise, your variables would reset.

`"messages" not in st.session_state` :

- This line checks if a key named `"messages"` exists in the `st.session_state` dictionary.
- **Purpose:** This `if` condition ensures that the initial chat history (the welcome message from the assistant) is only set **once** when the Streamlit application is first loaded or started by the user.
    - If the user interacts with the app (causing a rerun), `"messages"` will already be in `st.session_state`, and this `if` block will be skipped, preserving the existing chat history.

`st. session_state["messages"] =[ {"role":"assistant","content":"Hi,I'm a chatbot who can search the web. How can I help you?" } ]`

- This line is executed **only if** the `"messages"` key was not found in `st.session_state` (i.e., on the **very first run of the app** for a given user session).
- It initializes `st.session_state["messages"]` as a **list**.

> **Purpose:** This sets up the initial welcome message from the chatbot when a user first opens the application.

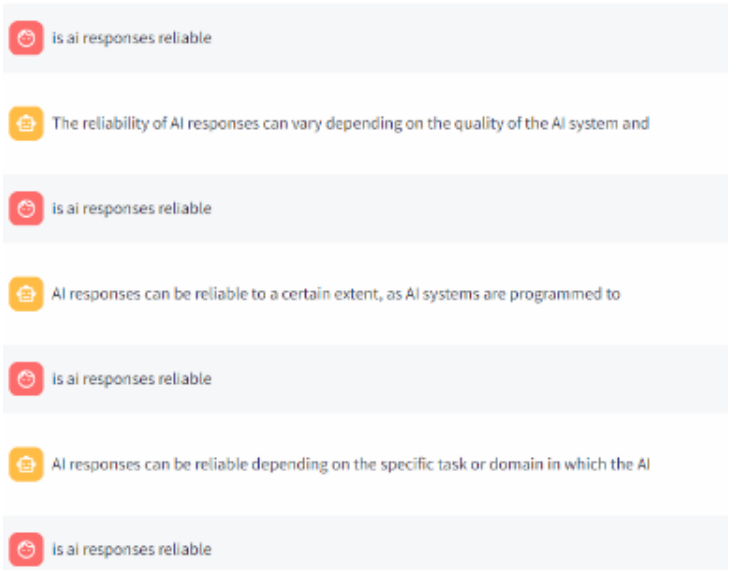
👇 After this:

Your chatbot will always start by saying:

> Hi, I'm a chatbot who can search the web. How can I help you?

```
for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg['content'])
```

- This loop iterates through every message dictionary stored in the `st.session_state["messages"]` list.
- **Purpose: This is how all the past messages (including the initial welcome message and any subsequent user/assistant exchanges) are retrieved for display.**
- It creates a "chat bubble" or a message container in the UI.
- The `role` argument (e.g., "assistant" or "user") is crucial because Streamlit will automatically style the message bubble differently based on the role (e.g., assistant messages on one side, user messages on the other, with appropriate avatars/colors).
- **`.write(msg['content'])`**: After creating the chat message container, the `.write()` method is used to display the actual text content of the message within that bubble.
- **Purpose:** This line, executed for each message in the `st.session_state.messages` list, renders the entire chat history on the Streamlit web page, presenting it as a visually appealing conversation.




```
if prompt:=st.chat_input(placeholder="What is machine learning?"):
    st.session_state.messages.append({"role":"user","content":prompt})
```

```
st.chat_message("user").write(prompt)
```

`st.chat_input(placeholder="What is machine learning?")`:

- `st.chat_input()`: This is a Streamlit widget specifically designed for chat interfaces. It renders a text input field at the bottom of the Streamlit app, similar to what you see in messaging applications.

- `placeholder="What is machine learning?"`: This sets the placeholder text that appears in the input field when it's empty, giving the user a hint about what they can type.

Return Value:

- If the user **has not yet typed anything and pressed Enter/clicked send**, `st.chat_input()` returns `None`.
- If the user **has typed something and pressed Enter/clicked send**, `st.chat_input()` returns the **string content** of what the user typed.

`if prompt:= ...`:

- This is a Python 3.8+ feature called a **"walrus operator"** (`:=`).
- It does two things simultaneously:
  1. It **assigns** the return value of `st.chat_input()` to the variable `prompt`.
  2. It **evaluates the assigned value as a boolean condition**.

> If they haven't typed anything, `prompt` is `None`, and the block doesn't run.

💡 **Effectively**: This `if` statement checks if the user has submitted a non-empty message. If `st.chat_input()` returns `None` (no input), the `if` condition is `False`, and the block is skipped. If `st.chat_input()` returns a string (user input), the `if` condition is `True`, and the `prompt` variable now holds that string.

`st.session_state.messages.append({"role":"user","content":prompt})` :

- This **adds the user's message** to the `messages` list in `st.session_state`.
- It **appends** a **new dictionary** **to the** `st.session_state.messages`
- `{"role": "user", "content": prompt}`: This matches the format used earlier, allowing the app to **track who said what**.

`st.chat_message("user").write(prompt)` :

- This **immediately displays** the user's message in the chat UI.
- `st.chat_message("user")` creates a **user-style chat bubble**.
- `.write(prompt)` puts the message inside it.

Where does `"message"` come from?

```
if "messages" not in st.session_state:
    st.session_state["messages"] = [
        {"role": "assistant", "content": "Hi, I'm a chatbot. How can I help you?"}
    ]
```

- This line checks whether `messages` exists in the Streamlit app's session memory, and **initializes** it if not.

Initialize a LangChain agent using Groq's LLM and a set of tools

```
llm=ChatGroq(groq_api_key=api_key,model_name="Llama3-8b-8192",streaming=True)
    tools=[search,arxiv,wiki]

    search_agent=initialize_agent(tools,llm,agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,handling_parsing_errors=True)
```

`streaming=True`: This is an important parameter. When set to `True`, the LLM will return tokens as they are generated, rather than waiting for the entire response to be completed.

This is crucial for:

- **Better User Experience**: In chat applications, users see the response building up in real-time, making the application feel more responsive.
- **Integration with Callbacks**: It works seamlessly with LangChain's callback system (like `StreamlitCallbackHandler`), allowing you to display the streamed output in your UI.

`tools=[search,arxiv,wiki]`

- This defines a list of tools (functions or utilities) the agent can **choose to use** while answering questions.
- Each of these tools conforms to the LangChain `Tool` interface (they expose a `.name`, `.description`, and `.invoke()` method under the hood).

```
search_agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    handle_parsing_errors=True
)
```

🔷 What this does?

- This uses `initialize_agent()` to create an **intelligent agent** that:
  - Uses your LLM (`llm`) as its **reasoning brain**
  - Chooses from the tools in the `tools` list
  - Uses a specific agent behavior style: `ZERO_SHOT_REACT_DESCRIPTION`

🧠 What is `AgentType.ZERO_SHOT_REACT_DESCRIPTION`?

This is a **reasoning style** where the agent:

1. Sees the user's question.
2. **Thinks aloud** (like "I need to look up this topic on Wikipedia").

3. Picks the right tool.
4. Uses the tool.
5. Gets the result.
6. Generates a final answer using that tool output.

It uses the **ReAct pattern** (Reason + Act), without needing example demonstrations — hence "zero-shot."

🛡️ What is `handle_parsing_errors=True`?

This tells the agent:

> If the model outputs something malformed (like JSON that can't be parsed), don't crash — try to recover and keep going.

This is important because LLMs sometimes generate unexpected outputs.

AI Response

```
with st.chat_message("assistant"):
    st_cb=StreamlitCallbackHandler(st.container(),expand_new_thoughts=False)
    response=search_agent.run(st.session_state.messages,callbacks=[st_cb])
    st.session_state.messages.append({'role':'assistant',"content":response})
    st.write(response)
```

This block:

1. **Calls the agent** (`search_agent`) with user input stored in `st.session_state.messages`
2. **Streams the agent's "thought process"** to the UI using `StreamlitCallbackHandler`
3. **Displays the final response** inside a chat bubble
4. **Stores the assistant's response** in the chat history

`with st.chat_message("assistant"):`

- Creates a chat bubble styled as if the assistant is speaking.

- Everything inside this block is displayed inside that bubble.

`st_cb= StreamlitCallbackHandler (st.container(),expand_new_thoughts=False)` :

> 💡 **This sets up a Streamlit callback handler to show the agent's step-by-step thoughts and tool usage in the UI.**

`st.container()` : This is a Streamlit command that creates an empty container. The `StreamlitCallbackHandler` uses this container to display the agent's intermediate "thoughts" (e.g., "Entering new AgentExecutor chain...", "Invoking tool...", "Tool output..."). By passing `st.container()` , you ensure these debug/trace messages are neatly organized within a specific area of your Streamlit app, usually below the "Agent thinking..." spinner and above the final answer.

`expand_new_thoughts=False` keeps tool-use steps *collapsed* by default for cleaner output.

```
response=search_agent.runcallbacks=[st_cb](st.session_state.messages,)
st.session_state.messages.append({'role':'assistant',"content":response})
```

`response = search_agent. run (st.session_state.messages, callbacks=[st_cb])`

- This is where the **AI does its work**:
  - `search_agent` : Your smart assistant (already set up earlier)
  - `run()` : Makes it answer based on your chat history

    > **👉 "Here's the chat so far (conversation history). Think carefully, use any tools you need, and give me the answer."**

    It sends the **entire chat history** stored in `st.session_state.messages` .
  - `st.session_state.messages` : Remembers all your past questions and its answers

- `callbacks=[st_cb]`: Connects the thought tracker
  - 👉 Show the step-by-step reasoning while the AI is thinking.

`st.session_state.messages.append({'role':'assistant', 'content': response})`

- After the assistant generates a reply (`response`), this line:

  👉 Adds that reply to the **chat history** (so we remember it for next turns).

- `role: assistant` → this is from the assistant
- `content: response` → this is what the assistant said

`st.write(response)` → Simply display the final response (AI's answer) on the screen under the assistant's message.

Entire Code:

```
import streamlit as st
from langchain_groq import ChatGroq
from langchain_community.utilities import ArxivAPIWrapper,WikipediaAPIWrapper
from langchain_community.tools import ArxivQueryRun,WikipediaQueryRun,DuckDuckGoSearchRun
from langchain.agents import initialize_agent,AgentType
from langchain_community.callbacks.streamlit import StreamlitCallbackHandler

## Arxiv and wikipedia Tools
arxiv_wrapper=ArxivAPIWrapper(top_k_results=1, doc_content_chars_max=200)
arxiv=ArxivQueryRun(api_wrapper=arxiv_wrapper)

api_wrapper=WikipediaAPIWrapper(top_k_results=1,doc_content_chars_max=200)
wiki=WikipediaQueryRun(api_wrapper=api_wrapper)

search=DuckDuckGoSearchRun(name="Search")


st.title("🔎 LangChain - Chat with search")
"""
```

```
In this example, we're using `StreamlitCallbackHandler` to display the thoughts and actions of an agent in an interactive Streamlit app.
Try more LangChain 🤝 Streamlit Agent examples at [github.com/langchain-ai/streamlit-agent](https://github.com/langchain-ai/streamlit-agent).
"""

## Sidebar for settings
st.sidebar.title("Settings")
api_key=st.sidebar.text_input("Enter your Groq API Key:", type="password")

if "messages" not in st.session_state:
    st.session_state["messages"]=[
        {"role":"assistant","content":"Hi,I'm a chatbot who can search the web. How can I help you?"}
    ]

for msg in st.session_state.messages:
    st.chat_message(msg["role"]).write(msg['content'])

if prompt:=st.chat_input(placeholder="What is machine learning?"):
    st.session_state.messages.append({"role":"user","content":prompt})
    st.chat_message("user").write(prompt)

    llm=ChatGroq(groq_api_key=api_key,model_name="Llama3-8b-8192",streaming=True)
    tools=[search,arxiv,wiki]

    search_agent=initialize_agent(tools,llm,agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,handling_parsing_errors=True)

    with st.chat_message("assistant"):
        st_cb=StreamlitCallbackHandler(st.container(),expand_new_thoughts=False)
        response=search_agent.run(st.session_state.messages,callbacks=[st_cb])
        st.session_state.messages.append({'role':'assistant',"content":response})
        st.write(response)
```